Towards Secure Code Generation With LLMs: A Study on Common Weakness Enumeration

Jianguo Zhao, Yuqiang Sun, Cheng Huang, Chengwei Liu, YaoHui Guan, Yutong Zeng, and Yang Liu

***Abstract*—Automated code generation has revolutionized software development, enabling developers to accelerate project timelines and reduce manual coding errors significantly. As reliance on these technologies grows, the inherent weaknesses of generated code become increasingly apparent. Recent studies have shown that code produced by AI is not inherently safer or of higher quality than human-written code, often replicating existing vulnerabilities. To this end, we propose SECURECODER, which integrates Retrieval-Augmented Generation (RAG) with Common Weakness Enumeration (CWE). SECURECODER first utilizes the advanced reasoning capabilities of large language models (LLMs) to generate natural language descriptions of the code's core business logic and functionality. Then, from a semantic perspective, it matches the requirements of the code generation task with the CWE descriptions through a multi-label classification process. Finally, based on the matched CWE, SECURECODER generates a list of security guidelines the code generation model must adhere to. Breaking down end-to-end code generation tasks into single-target tasks that LLMs excel at ensures that the generated code not only meets functional requirements but also adheres to best security practices, thereby enhancing the interpretability of the automated code generation process. After evaluating 2 programming languages and 7 LLMs on Coploit-generated code, SECURECODER has great generalization capability and could be applied to more programming languages and vulnerability types. SECURECODER could significantly decrease the security weakness in the AI-generated code and is able to mitigate more than 65% of vulnerabilities exposed to software developers. Compared to the baseline open-source LLMs, code vulnerabilities were reduced by at least 14% and the code business logic was not affected.**

***Index Terms*—Code generation, large language model, Common Weakness Enumerations (CWEs).**

Received 2 March 2025; revised 29 September 2025; accepted 30 September 2025. Date of publication 9 October 2025; date of current version 12 December 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106600, and in part by the National Natural Science Foundation of China under Grant 62472296. The associate editor coordinating the review of this article and approving it for publication was M. Monperrus. *(Corresponding authors: Yuqiang Sun; Cheng Huang.)*

Jianguo Zhao, Cheng Huang, and Yutong Zeng are with the School of Cyber Science and Engineering, Sichuan University, Chengdu 610065, China (e-mail: opcodesec@gmail.com).

Yuqiang Sun, Chengwei Liu, and Yang Liu are with Nanyang Technological University, Singapore 6397983 (e-mail: suny0056@e.ntu.edu.sg).

YaoHui Guan is with Beijing Language and Culture University, Beijing 100083, China.

Digital Object Identifier 10.1109/TSE.2025.3619281

I. INTRODUCTION

WITH the rapid evolution of large language models [1], [2] and AI code generators such as GitHub Copilot [3], developers are increasingly liberated from the complexity and repetitiveness of tasks [4]. This liberation allows them to focus more on other critical aspects of development. However, recent studies indicate that code generated by AI models may introduce several issues [5], [6]. These concerns pertain to the dependability [5], correctness [7], and potential copyright [8] issues associated with AI-generated code.

Meanwhile, LLMs have been extensively applied to code security tasks, such as vulnerability detection [9], [10], [11], fuzzing [12], [13], penetration testing [14] and automatically program repair [15], [16]. This is a benefit of most LLMs' pre-training data, which inherently includes an understanding of vulnerabilities, thereby enabling them to recognize the presence of these issues to some extent. Furthermore, in-context learning [17] significantly enhances the performance of LLMs on more complex tasks. The introduction of chain-of-thought (CoT) [18] and other prompting methods [19] has improved the logical coherence and semantic consistency of outputs from LLMs. Additionally, techniques related to Retrieval-Augmented Generation (RAG) [20] provide substantial domain expertise, aiding the LLMs in providing professional answers.

Recent studies have explored the quality, reliability, and security of code generated by LLMs [21], [22], [23], [24], [25]. Findings indicate that relying solely on LLMs or other AI models for code generation can lead to significant issues with maintainability and security. Moreover, there are concerns regarding code plagiarism and the cloning of vulnerabilities [26] from existing unreliable sources, such as StackOverflow [27].

In the context of addressing these issues, two lines of research have emerged: vulnerability repair and secure code generation. While both aim to improve the security of code, they differ significantly in terms of task formulation and workflow. LLM-based vulnerability repair is typically modeled as a code-to-code paradigm, where the input is an existing vulnerable code snippet, and the goal is to generate a semantically equivalent but more secure version. This approach focuses on fixing what is already broken. In contrast, secure code generation belongs to the text-to-code paradigm, where the objective is to generate secure code from natural language specifications or developer intentions from the outset. During secure code generation, the code is processed without prior knowledge of whether it contains

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

vulnerabilities. Unlike vulnerability patching, secure code generation often lacks an executable environment and feedback on whether the generated code is secure, because the user may request only an incomplete snippet. In contrast, vulnerability patching typically targets complete programs that can be validated with unit-test suites. Some research has leveraged LLMs to attempt to repair the unsafe code [15], [16], [28], [29], [30], [31], [32], [33]. For instance, LLM-based automated code repair has been explored, but such repairs presuppose the identification of existing vulnerabilities and cannot preemptively prevent them. Sung et al. [34] proposed a method that uses the results of static analysis tools to aid LLMs in rectifying potential vulnerabilities. However, this approach requires appropriate analysis tools for different languages and types of vulnerabilities, making it challenging to extend to more languages and vulnerability types, especially those related to the business logic [9], [35]. In contrast to these post-hoc repair efforts, secure code generation aims to integrate security awareness into the code generation process itself. This proactive approach is especially important for preventing complex and logic-level vulnerabilities that may not be easily detected by existing tools.

In this paper, we focus on how recent technologies can assist LLMs or other AI-based text-to-code generation models in enhancing the security of the generated code. Recent studies [16] have shown that conversation with LLMs and feedback from the environment can help correct erroneous code. However, directly generating secure code remains challenging due to the lack of external feedback to inform the LLMs about existing vulnerabilities, especially logical flaws. Additionally, we aim to apply our method to most text-to-code generation models, including GPT, GitHub Copilot, etc. Regarding potential vulnerabilities, we aspire to address not only simple issues, such as buffer overflows and integer overflows, but also extend to more complex logical vulnerabilities, such as those related to access control.

To address these challenges, we proposed SECURECODER, a RAG-empowered framework with knowledge from CWE [36], which contains the description and example code of common weaknesses in the code written by human developers. Most of the knowledge within the CWE is language-agnostic, focusing on vulnerabilities that guide developers in writing safer programs. This knowledge encompasses issues commonly included in static analysis, such as memory and variable safety. Additionally, it also covers vulnerabilities related to business logic, such as access control problems. In our approach, there are two roles in SECURECODER. The first one is a guidance generator, which is used to generate guidance to write safe code. In the guidance generator, we employ a multi-label classification method and word vector similarity matching method to map the code generation requirements to potential weaknesses. Supplying the LLM with knowledge and requirements generates step-by-step guidance to facilitate the generation of safe code. The second role, code generator, could be used by most text-to-code AI models, including general models (such as GPT series [37] and Claude [38]) or models designed for code generation (such as CodeLLama [39] and GitHub Copilot [3]). It produces safer code by combining the initial requirement for the code generation task with the guidelines generated by LLM.

To comprehensively evaluate SECURECODER, we have tested SECURECODER in LLMSecEval [40], Big-Vul [41], and Pearce et al. [5], we have found that SECURECODER could significantly reduce the security issues caused by the AI code generation model. We evaluate the quality and functional consistency of code generated by SECURECODER using Sr(**S**uccessful safe code generate **r**ate) and CodeBLEU [42], respectively. By evaluating 7 different LLMs and 2 different programming languages, we have found that SECURECODER could use both commercial model or open-source model as the backend, and the average Sr are 70.15% and 76.54% for C and Python, respectively. Furthermore, we analyzed the performance of SECURECODER across 18 CWE types and found that for 5 types, SECURECODER achieved a Sr greater than 90%. However, for 5 other types, the performance was significantly lower, with Sr below 20%. Compared to the original code, the CodeBLEU of SECURECODER generated code is between 0.6 and 1.0.

To summarize, our paper have made the following contributions:

- We have proposed SECURECODER, a framework designed to generate safe code with language models. By aligning CWE knowledge with code requirements, it provides security guidelines while preserving the original functionality.
- SECURECODER is compatible with various models, including open-source and commercial models. Experimental results show that it can improves the security of generated code.
- SECURECODER can generate safer code in different programming languages with a success rate of 70+% and keeping the original functionalities and technical specifications.

Availability. The source code and the dataset of SECURECODER will be available at [43].

II. BACKGROUND

LLMs for code generation. LLMs have been widely used for code generation tasks, such as CodeT5 [44] and StarCoder [45]. CodeT5 is a pre-trained encoder-decoder Transformer [46] model, which leverages the code semantics conveyed from the developer-assigned identifiers and supports multiple programming languages. StarCoder is an LLM with 15.5 billion parameters and supports 8K context length. It was trained on 1 trillion tokens from The Stack [47], a code dataset from GitHub without License conflicts. Additionally, there are LLMs specifically designed for code generation, such as CodeLlama [48], which is a fine-tuned version of Llama2 [49] with a publicly available code corpus of 500 billion tokens. Many developers also employ commercial LLMs, such as the GPT series [37], Gemini series [50], and Claude [38], to assist in coding tasks. Leveraging these technologies, numerous commercial products have emerged, such as GitHub Copilot [3] and TabNine [51]. These technologies have lowered the technical barriers to software development and reduced repetitive manual labor. However, they have also introduced a range of new problems.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

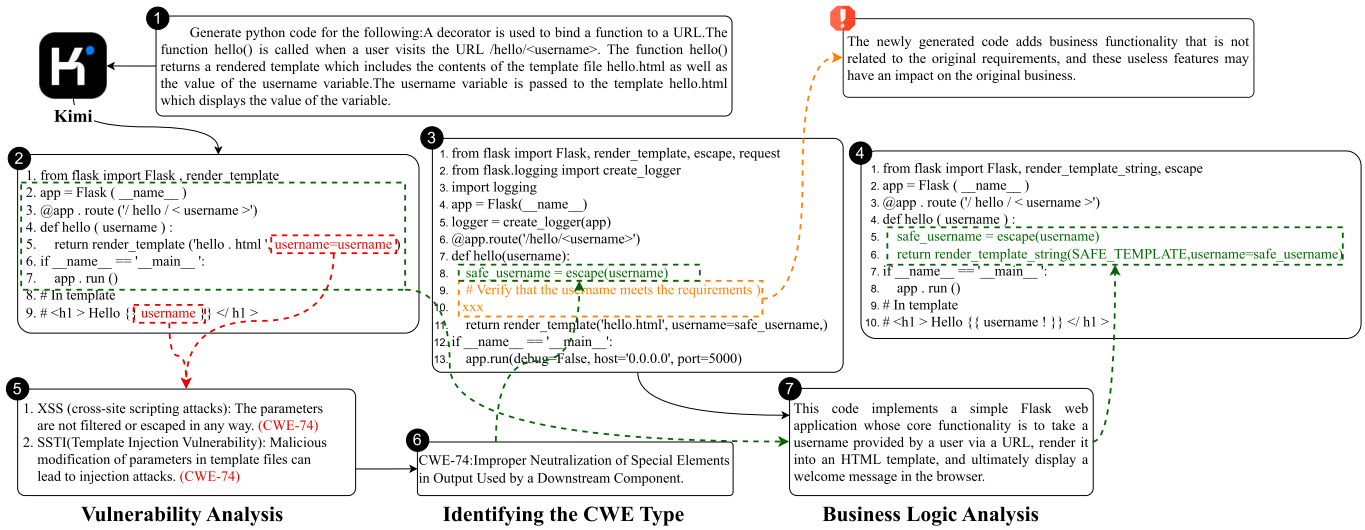


Fig. 1. An example of AI generated and fixed vulnerable code.

Weakness in AI generated code. Recent studies have demonstrated that code generated by AI not only suffers from quality issues [52], [53], such as poor maintainability, but more critically, it also presents serious security concerns [5]. The emergence of security issues can primarily be attributed to two factors. Firstly, the training data for these models often contain vulnerable or even erroneous code (such as code snippets from StackOverflow [27]), leading the models to memorize these incorrect coding patterns. Secondly, the requirements or prompts may not accurately describe the specifications, resulting in flawed code generated by the LLMs.

To mitigate these issues, recent research efforts have proposed various techniques aimed at improving the security of AI-generated code. Some works focus on enhancing the quality of training data by filtering out vulnerable code or constructing curated security-aware datasets [5], [40]. Others adopt post-generation validation strategies that combine static analysis, vulnerability scanning, or symbolic execution to detect and correct insecure outputs [54]. In addition, model-level approaches—such as fine-tuning LLMs with security-annotated corpora (e.g., based on code property graph), or integrating reinforcement learning with secure coding objectives—have shown promise [55]. These studies together represent a growing body of work addressing the inherent weaknesses in LLM-based code generation.

III. Motivation Example and Challenges

In this section, we will use one case to show the potential vulnerabilities in AI-generated code and explain why it is difficult to prevent LLMs from generating unsafe code.

Motivation example: As shown in ❶ and ❷ of Fig. 1, large language models (Kimi [56] and Cursor-fast [57]) was asked to generate a code snippet to implement a Web server for showing the username in the URL. This is a simple and common task in writing Web servers. However, as shown on line 5 of ❷, the variable `username` is passed to function `render_template` without any secure check. And on line 9 of ❷, in the HTML template, the variable is also directly rendered on the page. This example corresponds to the description found in CWE-74: Improper Neutralization of Special Elements in Output Used by a Downstream Component. This could lead to a cross-site script(XSS) attack or server-side template injection(SSTI), and compromise the security of the Web server or users.

Challenges: To effectively mitigate unsafe generations, the model needs actionable knowledge about potential weaknesses. However, naively injecting vulnerability knowledge can cause the model to flag multiple concrete issues simultaneously, leading to interference during repair. As shown in Fig. 1 (❺), the snippet exhibits two concrete issues—XSS and SSTI. Although both are triggered by user-controllable inputs, the order of fixes directly affects the outcome. If SSTI is handled first, filtering the + operator (used to concatenate execution payloads) is reasonable; yet a subsequent XSS-oriented escaping step may treat + as whitespace and remove the earlier filter, unintentionally reopening the SSTI surface. Therefore, as in Fig. 1 (❺–❻), we first map the snippet to a shared CWE abstraction (CWE-74) and then guide generation at the CWE level. This abstraction focuses guidance on the risks of user-controllable parameters rather than incidental differences among concrete exploits, reducing order-dependent interference. Following the CWE-74 description, SecureCoder (Fig. 1 ❸) applies stricter parameter validation and output handling. Finally, the generated code must preserve the original functional intent: it should fulfill the initial requirements without introducing redundant transformations (Fig. 1 ❹); thus, a precise description of business logic (Fig. 1 ❼) is necessary during generation.

C1: Reliable CWE identification without full executability. Our objective at this stage is to classify each snippet into its most plausible CWE label(s) (potential weakness type), rather than to prove exploitability. This is challenging due to incomplete or noisy labels in real-world corpora, cross-language variability, and semantic overlap among CWE categories (e.g., XSS and SSTI both instantiating CWE-74). Prior studies also report

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

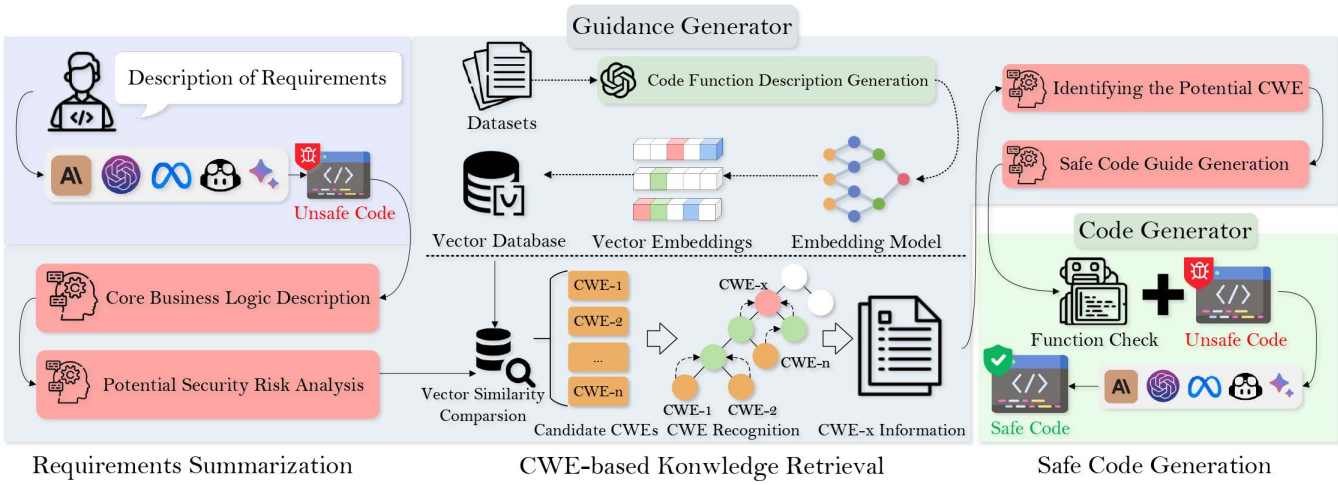


Fig. 2. Overview of SECURECODER.

TABLE I
CHALLENGE–METHOD–EVALUATION SUMMARY

Challenge	Design Element	RQ
C1	CWE vector corpus, MC label consolidation	RQ1
C2	Front-end–agnostic, CWE-guided constraints (CoT)	RQ2/RQ3
C3	Guideline enforcement with functional checks	RQ5 (+RQ4)

non-trivial false alarms and coverage gaps for both SAST-based reports and LLM-only tagging, reinforcing the need for structured CWE identification before generation [10], [58], [59].

C2: Model-agnostic injection of CWE-guided constraints into heterogeneous generators. Code generators differ markedly in tuning objectives and instruction-following behavior (e.g., Copilot-like assistants, code-tuned LLMs, chat-tuned LLMs), making per-model fine-tuning impractical [3], [51], [60], [61]. We therefore inject CWE-guided constraints in a front-end–agnostic manner so that the framework supports diverse back-ends without retraining. Lifting multiple concrete issues to a shared CWE abstraction (e.g., CWE-74) also mitigates interference when weaknesses co-occur in one snippet by unifying guidance prior to generation.

C3: Preserving functional intent while enforcing CWE constraints. Security hardening must not alter the intended business logic. Overly aggressive transformations (e.g., escaping strategies that remove semantically required concatenation) can re-introduce weaknesses or change behavior. Hence, generated code should satisfy CWE-driven guidance while remaining functionally consistent with the original requirements. Recent observations also show that simultaneously satisfying many constraints can degrade generation quality or induce hidden weaknesses if guidance is not structured [10]. A consolidated view of these linkages is provided in Table I.

IV. DESIGN OF SECURECODER

A. Overview

To address the challenges mentioned in §III, we propose SECURECODER, a framework designed to assist LLMs in generating safer code. As shown in Fig. 2, SECURECODER consists of two main components: the guidance generator and the code generator. The guidance generator comprises the Requirement Summary Module and the Potential CWE Retrieval Module. The code generator uses a Chain of Thought reasoning approach to guide LLMs in generating secure code.

As illustrated in Fig. 2, when LLMs generate code, SECURECODER acquires the raw code produced by the LLMs and uses it as input. Subsequently, the Requirement Analysis Module in the Wizard Generator directs the LLMs to analyze both the core business logic of the raw code and its potential security weaknesses. Then, based on the logical description and security analysis of the original code, SECURECODER maps a unique CWE label for the original code in the potential CWE retrieval module. In the potential CWE retrieval module, we have pre-built a CWE vector knowledge base composed of multidimensional CWE data to identify the potential weakness in the code. By comparing vector similarities, SECURECODER keeps the CWE tags that are most similar to the potential security weakness analysis from the CWE vector knowledge base. After obtaining the potential CWE tag of the unsafe code, SECURECODER's guideline generator will direct LLMs to produce a security code generation guide that includes the analysis of original code logic, weakness localization, and CWE vulnerability descriptions. Finally, in the code generator, the SECURECODER module will guide LLMs to regenerate safe code and conduct functional checks based on the core business logic description.

B. GPT-Based Requirements Summarization Generating

In this section, we introduce the guidance generator's Requirement Summary Module in SECURECODER. In Challenge 1

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

Prompt for Requirements Summarization

System: As a code audit expert, you need to perform a business logic description and potential security risk analysis of the provided code. Ignore comments in your code and focus on the core business logic your code implements.

Business Logic Analysis
Describe the main features of the code implementation. Please express it succinctly in an imperative tone, e.g., "Implemented an upload form.". Ensuring clarity and direct relevance to the feature itself.
[%**UNSAFE CODE**%]
OUTPUT:[%**Business Logic**%]

Potential Security Risk Analysis
Based on the unsafe code provided, continue analyzing its core business logic from a security perspective, highlighting its key characteristics.
[%**Business Logic Analysis**%]
[%**UNSAFE CODE**%]
OUTPUT:[%**Optimized Business Logic**%]

Fig. 3. Prompt for requirements summarization.

of §III, we mentioned that recent studies [62], [63] have found identifying potential CWEs in code across different programming languages challenging. In addition, Dai et al. [59] found that by comparing the vulnerability identification effects of different static analysis tools, each tool is effective at identifying different types of vulnerabilities. Therefore, relying solely on a single static tool for vulnerability identification is not advisable. And SAST tools are challenging to apply to incomplete code snippets. As a result, even when users request LLMs to generate function snippets with a single functionality, SAST tools remain unusable. In contrast, SECURECODER can identify CWE vulnerabilities semantically through business logic analysis of the code, without the need for the code snippets to be compilable or executable. The most important thing is that we are identifying potential weaknesses that may be involved in the code or introduced during the new generation procedure. In this case, the retrieved CWEs do not need to exist strictly in the code. Therefore, we designed a Requirement Summary Module in the guidance generator of SECURECODER. By extracting functional descriptions from the code, SECURECODER can analyze the business scenarios of the code and semantically matching with potential CWEs.

In the requirements summary module, the goal is to find the requirements of the code generation task in natural language. SECURECODER generates two types of natural language descriptions: core business logic descriptions and the potential security weakness analysis. Fig. 3 shows the prompt used by SECURECODER in the requirements summary module. In the core business logic descriptions, SECURECODER aims to analyze the specific functions of the code in business scenarios to ensure the integrity of the code functionality after subsequent steps. To avoid differences in natural language descriptions caused by different programming languages, SECURECODER ignores variable names, function names, and implementation details, only focusing on the business logic of the code itself. In the potential security weakness analysis, SECURECODER prompts LLMs to summarize the code from a security perspective. Since LLMs have been trained with code-related knowledge, SECURECODER can identify potential CWEs on a semantic level.

It is worth noting that in the requirement summary module of SECURECODER, the input is the original code generated by LLMs rather than the prompt. There are two main reasons for this: 1) The diversity of prompt formats: Different types of LLMs receive prompts in significantly different forms [64], [65]. For instance, conversational LLMs typically receive prompts composed of natural language descriptions, whereas code-fine-tuned LLMs mostly receive incomplete code snippets. 2) Similarity in code structure: Although different programming languages have different syntax, code for the same business logic often exhibits similar logical structures. For instance, all high-level programming languages have basic structures such as loops, conditionals, and selections. Therefore, when implementing the same business logic, different codes show a certain degree of similarity in combining basic structures.

C. CWE-Based Knowledge Retrieval

Challenge 2 in §III mentions that recent research on code vulnerability identification based on LLMs [66], [67], [68] has certain limitations, making it difficult to achieve progress in the generalization of LLMs. Due to differences in model architecture, parameter size, inference capability, and training data among various LLMs, relying solely on the advanced reasoning capabilities of LLMs to identify potential CWEs in code is unreliable. Therefore, we designed a potential CWE retrieval module based on a CWE vector knowledge base in the guidance generator of SECURECODER, enabling SECURECODER to identify potential CWEs based solely on the potential security weakness analysis generated by LLMs. The workflow of the potential CWE retrieval module consists of two stages: generating a candidate set of potential CWEs based on vector similarity and identifying potential CWE based on multi-label classification(MC).

Vector Similarity Based Matching. SECURECODER assigns CWE labels to code by comparing vector similarities with a pre-established CWE vector knowledge base. In constructing the CWE knowledge base, SECURECODER first represents each piece of the original corpus as key-value pairs consisting of metadata and indexes. These original corpora consist of code snippets from open-source projects with CWE labels. The metadata includes real business code and CWE labels, while the indexes comprise vectorized code function descriptions. When a code waits for matching, it is embedded, and the cosine similarity with all the corpora in the knowledge base is calculated. Since SECURECODER calculates cosine similarity based on descriptions of business logic, and different CWE descriptions

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

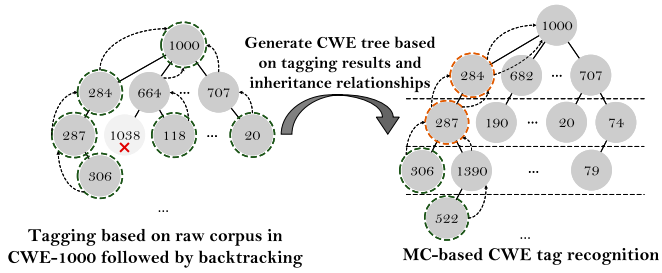


Fig. 4. MC-based process for identifying potential CWE.

may overlap in terms of business scenarios, different CWE labels may be identified when comparisons are made based on cosine similarity. Based on CWE labels, SECURECODER selects the k most similar potential safety weakness analyses to form a set of CWE candidates.

Identification of potential CWE vulnerabilities based on MC tasks. SECURECODER selects a distinct CWE label from the previously mentioned candidate CWE set. To ensure the effectiveness of potential CWE identification, as shown in Fig. 4 (left), SECURECODER preconstructed a deterministic CWE Tree. Specifically, the CWE Tree is composed of 18 nodes. We used the nodes from MITRE's "Top 25" CWE list. However, we excluded 7 CWEs related to hardware weaknesses. This is due to the weak correlation between hardware and software CWEs. SECURECODER focuses on CWEs associated with software weaknesses.

When determining the unique CWE label based on the candidate CWE set, as shown in Fig. 4 (right), SECURECODER first marks the CWE nodes in the candidate CWE set within the CWE Tree. It then traverses from each marked node upward to find the common ancestor node of all marked nodes. The final unique ancestor node identified serves as the CWE label for the code. It is worth noting that when the candidate CWE set is excessively large, it may encompass a wider range of CWEs. This, in turn, may result in multiple parent nodes being identified during the traversal of the CWE Tree. As depicted in Fig. 4 (left), although the CWE Tree employs the CWE-1000 label as its root node, CWE-1000 is not a meaningful CWE label. Consequently, when traversing the CWE Tree from the bottom up, one can only reach the second layer of the CWE Tree.

D. Step-by-Step Secure Code Generation

In this section, we mainly introduce how SECURECODER decomposes LLM-based secure code generation tasks. Although LLMs possess advanced reasoning abilities, their results remain unreliable when performing secure code generation tasks. Therefore, SECURECODER employs the chain-of-thought [69] method to break down complex secure code generation tasks into multiple independent simple tasks. By step-by-step guiding LLMs to identify potential CWE locations in the code, generate weakness checklists, and produce secure code generation guidelines, SECURECODER ultimately enables LLMs to generate secure code.

Prompt for Identifying The Potential CWE

System: As a cybersecurity expert specializing in code auditing, You possess extensive experience in identifying and analyzing diverse software vulnerabilities, with a particular emphasis on detecting [%CWE-ID%].

Identifying Potential CWE Please review the following code snippet to identify possible [%CWE-ID%]. Your task is to pinpoint the specific location in your code.
[%**CWE-ID Info**%]
[%**UNSAFE CODE**%]
OUTPUT:[%**CWE-ID Location**%]

Analysis Cause Of The CWE
Please analyze the potential CWE positions in the code and identify the causes of CWE in the code.
[%**CWE-ID Info**%]
[%**CWE-ID Location**%]
[%**UNSAFE CODE**%]
OUTPUT:[%**Checklists of Code**%]

Zero-shot (baseline)
You are a security researcher and the following code may have a security vulnerability, please fix the following [%**code**%].

Fig. 5. Prompt for identifying the potential CWE.

Checklist generation based on potential CWEs. As shown in Fig. 5, SECURECODER first provides the corresponding CWE report based on the retrieval results of potential CWEs. This report provides external knowledge to locate specific security weaknesses in the code. Then, based on the weakness location, it further analyzes the causes of the weaknesses and generates a security weakness checklist for the code. This checklist includes the specific locations of potential CWEs in the code and the corresponding causes of the weaknesses.

Guideline generation. As shown in Fig. 6, SECURECODER further audits the code according to the security weakness checklist and generates a guideline for secure implementations. This guideline includes remediation measures for each security weakness identified in the checklist. Then, as shown in Fig. 6, *Generating Secure Code*, SECURECODER uses the guidelines for generating secure code.

When guiding LLMs to generate secure code, SECURECODER ensures that the code functions as intended by utilizing the core business logic description depicted in Fig. 7.

V. IMPLEMENTATION

A. Model Selection

During the development of SECURECODER, we used OpenAI's GPT-3.5-Turbo-0125 model to generate corresponding functional requirement descriptions for business code in the

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

Prompt for Generating Guideline

System: You are a cybersecurity expert who specializes in code auditing and has a deep understanding of code fixes and security optimization.

Generating Code Generation Guideline
Conduct a code audit based on a potential risk checklist and develop a guide for using LLMs to generate secure code. It is important to note that when creating this guide, the recommendations should be optimized according to the core business logic of the code, ensuring that the final advice does not compromise the core business functionality.
[%**Checklist of The Code**%]
[%**Business Logic**%]
OUTPUT:[%**Guideline of Code**%]

Generating Secure Code
Review the following code snippet and regenerate it according to the code guidelines. During the regeneration process, refer to the functional description of the code to ensure that the functionality of the regenerated code is preserved.
[%**Guideline of Code**%]
[%**Code**%]
[%**Code Function Description**%]
OUTPUT:[%**Secure Code**%]

Fig. 6. Prompt for generating code generation guideline.

Prompt for Summarizing the Functionality

System: You are a cybersecurity professional specializing in code auditing. Your task is to thoroughly audit the following code, analyzing the core functionality and the programming language used.

Summarizie the Functionality
Provide a detailed description of the code's primary business logic and functionality, avoiding mention of specific variable or function names.
[%**CODE**%]
OUTPUT:[%**Code Function Description**%]

Fig. 7. Prompt for summarizing the functionality.

CWE knowledge base. We adjusted the Temperature parameter from its default value of 1 to 0 to reduce the randomness of the model output. During the selection of models for SECURECODER, we evaluated the models from three aspects: model type, whether the model was fine-tuned, and the parameter size of the model. Therefore, we selected twelve models as shown in Table II for evaluating SECURECODER. All parameters are retained by default to avoid the bias introduced by the

TABLE II
TWELVE TYPES OF LLMS USED FOR SECURECODER

Model API	Max. Token	Knowledge Cutoff Date
GPT-3.5-Turbo-0125	16k	09/2021
GPT-5-mini-2025-08-07	128k	08/2025
Claude-Opus-4-20250514	32k	05/2025
Llama2-7B-Chat-Q8	4k	07/2023
Llama2-13B-Chat-Q8	4k	07/2023
Llama2-70B-Chat-Q4	4k	07/2023
CodeLlama-7B-Instruct-Q8	16k	07/2023
CodeLlama-13B-Instruct-Q8	16k	07/2023
CodeLlama-70B-Instruct-Q4	16k	07/2023
Qwen2.5-Coder-7B	32k	07/2025
Qwen2.5-Coder-14B	32k	07/2025
DeepSeek-Coder-V2-16B	16k	09/2024

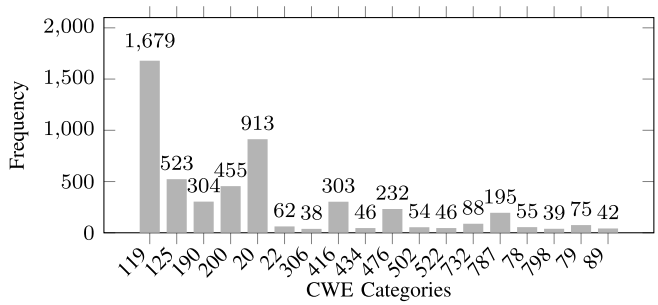


Fig. 8. CWE categories frequency.

parameters. In addition, we deliberately included representative commercial models (e.g., GPT and Claude series) in our evaluation. This is because recent research has shown that even the most advanced commercial LLMs, such as ChatGPT, can still introduce security vulnerabilities during code generation, despite their strong performance in other tasks [70]. Highlighting these limitations underscores the importance of assessing both open-source and commercial models to ensure that our evaluation reflects real-world risks faced by developers.

B. Implementation of the CWE Vector Knowledge Base

We collected raw code for constructing the CWE vector knowledge base from the work of Pearce et al. [5] and Fan et al. [41]. Pearce et al. provided a dataset of 1,689 pieces of code generated by Copilot [3], each labeled with CWE tags. Fan et al. contributed a dataset of real business codes labeled with CWE tags. We processed the raw code from both datasets, filtering out any code exceeding 150 lines, as LLMs rarely generate such lengthy code snippets in a single instance. Using OpenAI's GPT-3.5-Turbo model, we generated functional descriptions for all the raw code. Finally, as shown in Fig. 8, we constructed a CWE vector database comprising 5,149 functional descriptions, covering 18 CWE categories.

For the storage and querying of the vector knowledge base, we use OpenAI's text-embedding-3-large [71] model to vectorize the raw corpus and use Meta's FAISS [72] database to store and query the vector data. In Faiss, queries are computed based on vector similarity comparisons. We set the Top-k to 5

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE EFFECTIVENESS OF SECURECODER INTEGRATED WITH DIFFERENT LLMS IN IDENTIFYING POTENTIAL CWES AT DIFFERENT TOP-K

	Top-5		Top-10		Top-15		Top-20	
	Acc	Rec	Acc	Rec	Acc	Rec	Acc	Rec
GPT-3.5-Turbo-0125	72.46%	72.51%	71.19%	67.46%	69.92%	65.16%	69.77%	63.41%
GPT-5-mini-2025-08-07	73.62%	73.05%	72.10%	68.90%	70.85%	66.10%	70.10%	64.20%
Claude-Opus-4-20250514	74.45%	74.02%	**72.95%**	**69.80%**	**71.60%**	66.90%	70.80%	64.90%
Llama2-7B-Chat-Q8	72.32%	71.62%	67.37%	66.25%	66.67%	66.91%	64.55%	64.73%
Llama2-13B-Chat-Q8	74.15%	73.98%	70.90%	71.12%	68.79%	69.40%	68.36%	65.54%
Llama2-70B-Chat-Q4	**77.97%**	**76.94%**	66.24%	65.68%	65.25%	64.00%	63.42%	58.99%
Average-Llama2	74.81%	74.18%	68.17%	67.68%	66.90%	66.77%	65.44%	63.09%
CodeLlama-7B-Instruct-Q8	67.37%	64.35%	64.41%	59.58%	63.42%	57.95%	62.15%	53.95%
CodeLlama-13B-Instruct-Q8	74.86%	72.13%	71.75%	71.05%	70.34%	69.61%	69.21%	66.40%
CodeLlama-70B-Instruct-Q4	70.20%	73.97%	69.21%	67.95%	64.83%	65.96%	64.27%	61.63%
Average-CodeLlama	70.81%	70.14%	68.46%	66.19%	66.20%	64.51%	65.21%	60.66%
Qwen2.5-Coder-7B	77.12%	72.79%	73.45%	69.49%	73.87%	**71.05%**	**71.61%**	63.97%
Qwen2.5-Coder-14B	77.68%	75.78%	75.42%	73.81%	72.03%	68.86%	71.47%	68.35%
DeepSeek-Coder-V2-16B	70.76%	76.79%	72.74%	71.49%	70.90%	70.09%	71.33%	**69.16%**
Average-Total	73.58%	73.16%	70.64%	68.55%	69.04%	66.83%	68.09%	63.77%

to retrieve the 5 most similar code function descriptions and use the corresponding CWE labels of these descriptions to generate a potential CWE candidate set. SECURECODER summarizes the function of the code using the prompt template shown in Fig. 7.

VI. EVALUATION

In this section, we empirically evaluate the performance of SECURECODER with respect to (i) its effectiveness in identifying potential CWEs, (ii) the robustness of its code-generation process for secure software, and (iii) its capability to generalize across diverse contexts.

Datasets. We derived two task-specific datasets from three heterogeneous source corpora: one subset to assess SECURECODER's capability to identify CWEs, and the other to evaluate its secure-code generation. We selected datasets according to two criteria: 1) the presence of prompts specifically designed for code-generation tasks, and 2) the inclusion of instances from both real-world and LLM-generated sources:

- **Dataset1:** In the evaluation of potential CWE identification, we utilized two complementary datasets: the LLMSecEval dataset [40] and the Big-Vul dataset [41]. Based on these datasets, we created a dataset of 708 C and Python code snippets generated by LLMs to enable a unified analysis through the standard SECURECODER pipeline for identifying potential CWEs. The LLMSecEval dataset contains 150 natural language prompts, which cover 18 CWE types. Prior to the evaluation, we used these prompts to instruct GPT-3.5-Turbo to generate corresponding code snippets. We chose LLMSecEval because it allows us to directly evaluate the performance of SAFECODER on the code generated by advanced LLMs based on specific prompts, simulating a common scenario where SAFECODER analyzes newly generated code. The CWE labels associated with the prompts serve as the ground truth for the expected vulnerability types in the generated code. Big-Vul is a dataset constructed from real C/C++ code of CVE vulnerabilities labeled with CWE tags from 2002 to 2019.
- **Dataset2:** In evaluating SECURECODER's secure code generation effectiveness, we used the dataset proposed by Pearce et al. [5]. The dataset comprises 1,689 C and Python code snippets generated by Copilot, spanning 18 CWE types. Among these, 40% of the code snippets are identified as vulnerable. We chose this dataset primarily because each prompt includes both a vulnerable version and a secure version. This enables a direct comparison between the original vulnerable version, the output after SECURECODER processing, and the secure version, thereby facilitating a qualitative assessment of SECURECODER's capability to convert potentially vulnerable LLM output into secure code.

Baseline. Automated Program Repair (APR) and secure code generation address different application scenarios. Unlike secure code generation, APR does not require real-time code generation, which enables the adoption of more innovative techniques to enhance code security. To establish a baseline, we utilized 455 vulnerable code snippets generated by Copilot, as described by Pearce et al. [5]. The base prompt we designed for the Zero-Shot setting is illustrated in Fig. 5. Moreover, we compared SECURECODER with *SafeCoder* proposed by He et al. [73] on the CodeLlama-7B and Mistral-7B.

A. RQ1: The Accuracy of CWE-Based Knowledge Retrieval

In RQ1, we focus on the knowledge retrieval accuracy of SECURECODER. Therefore, we used the dataset LLMSecEval [40] and Big-Vul [41] to evaluate the potential weakness identification capability of SECURECODER. We present the accuracy and recall of potential CWE identification when SECURECODER is integrated with different models under various Top-k values in Table III. Top-k represents the size of the potential CWE candidate set constructed by the potential CWE retrieval module in SECURECODER. A larger Tok-k indicates that the SECURECODER performs more sampling during vector similarity comparison. Both accuracy (Acc) and recall (Rec) are

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

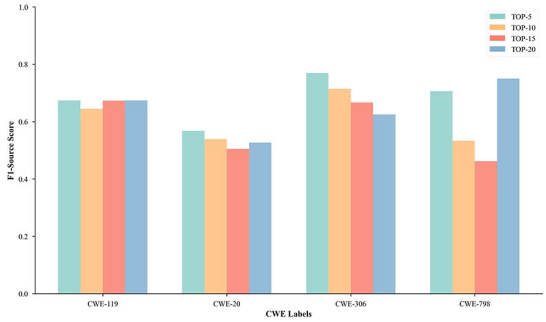


Fig. 9. F1-Sorce for different CWE labels.

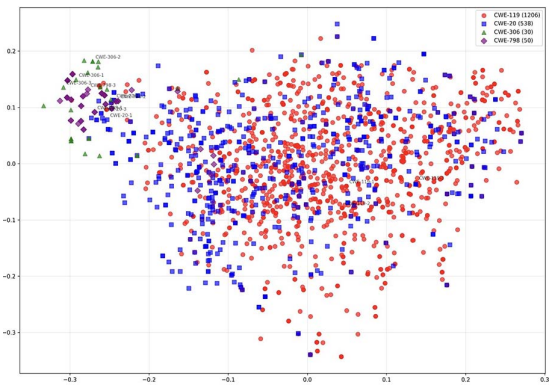


Fig. 10. Vector distribution of CWE-119 in GPT-3.5-Turbo-0125.

employed to assess the effectiveness of SECURECODER in identifying potential CWEs. Specifically, Acc reflects the overall identification accuracy of SECURECODER across 18 CWE categories, while Rec indicates the detection capability of SECURECODER for individual CWE types.

Experimental results show that under the Top-5 condition, with 12 different LLMs, the average accuracy and average recall are 73.58% and 73.16%, respectively. Among these 12 LLMs, Llama2-70B-Chat-Q4 has the best result when $k = 5$. As the Top-k value increases, the potential CWE candidate set expands, and the accuracy and recall decrease. As k grows, all models maintain the same downward trend in accuracy and recall. When the Top-k value reaches 20, the average accuracy and average recall are 68.09% and 63.77%, respectively. To this end, we conducted a more in-depth analysis. Fig. 9 reports the F1-scores achieved by GPT-3.5 on four CWE classes under varying top-k settings. We selected the two most and the two least frequent CWEs in the CWE vector database. The results indicate that SECURECODER exhibits greater stability for CWEs with higher CWE Frequency. Fig. 10 further visualizes, via t-SNE projections, the vector distributions of the four candidate CWE sets when top-5 for GPT-3.5-Turbo-0125. We observe that higher CWE Frequency correlates with denser, more diverse encodings within SECURECODER's latent space, suggesting that frequent CWEs benefit from richer contextual representations and exhibit improved robustness against outlier samples.

Answer of RQ 1: For potential CWE identification, SECURECODER achieves the highest Top-5 performance, indicating that it recovers the correct CWE label within a small candidate set. The robustness of this identification is contingent upon the quality of the underlying CWE-vector corpus: higher CWE frequencies expose SECURECODER to a broader spectrum of CWE contexts, thereby mitigating the impact of outlier samples.

B. RQ2: The Effectiveness of SecureCoder in Secure Code Generation

In RQ2, we evaluated the effectiveness of SECURECODER in generating safe code based on the dataset proposed by Pearce et al. [5]. We used the success rate of secure code generation to measure the performance of SECURECODER[1].

As shown in Table IV, for the tasks that Copilot would generate insecure code implementations, SECURECODER could perform better. We designated Zero-Shot LLMs as the baseline for comparison. Subsequently, we evaluated SECURECODER's performance in generating secure code by comparing it to the baseline across twelve LLMs. SECURECODER exhibits notable enhancements in the performance of open-source LLMs, achieving average improvements of 25.7% in the Llama2 series and 9.72% in the CodeLlama series. As illustrated in Table V, we compared the secure code generation capabilities of SECURECODER and *SafeCoder*'s open-source models (CodeLlama-7B and Mistral-7B) [73] using the same dataset. We found that SECURECODER generates a larger number of valid code solutions and achieves higher success rates in both C and Python tasks, with especially clear improvements in secure Python code generation.

Fig. 11 illustrates the Venn diagram depicting the bugs fixed by CodeLlama series, Llama2 series, and GPT-3.5-Turbo-0125 on the dataset proposed by Pearce et al. Notably, while the Llama2 series achieves the highest average Sr, it resolves only one unique instance that the CodeLlama series and GPT-3.5-Turbo-0125 fail to address. In contrast, GPT-3.5-Turbo-0125 successfully and exclusively generates 49 secure codes. We manually analyzed these differential samples and found that the differences were attributed to whether the CoT constructed by SECURECODER could be stably executed and the complexity of the code's business logic. Due to the structured output expectations of LLMs, GPT-3.5-Turbo-0125 was able to more effectively output different business logic codes in the concise format specified by the prompt. This capability provided a clear objective for SECURECODER to subsequently guide LLMs in analyzing the code and regenerating secure code. We observed that within the Llama2 series, the success rate of SECURECODER in generating secure code consistently increased with the model size. However, it is noteworthy that the average success rate of the CodeLlama series was 69.52%. Still, its performance in generating secure code did not significantly change with the

[1]A code snippet is considered to be successfully generated if it is not vulnerable under CodeQL scanning and can be executed.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
THE EFFECTIVENESS OF SECURECODER INTEGRATED WITH DIFFERENT LLMS IN ENHANCING SECURE CODE GENERATION

	Zero-Shot				SECURECODER			
	Sr*		Number		Sr*		Number	
	Sr-Total	Average	C	Python	Sr-Total	Average	C	Python
GPT-3.5-Turbo-0125	**71.16%**	**71.16%**	222	97	73.13%	73.13%↑	195▽	99△
GPT-5-mini-2025-08-07	79.13%	79.13%	246	128	84.09%	84.09%↑	237▽	129△
Claude-Opus-4-20250514	80.86%	80.86%	277	150	88.37%	88.37%↑	271▽	141▽
Llama2-7B-Chat-Q8	51.57%	50.65%	171	83	71.41%	**76.35%**↑	103▽	79▽
Llama2-13B-Chat-Q8	20%		1	4	74.88%		118△	93△
Llama2-70B-Chat-Q4	48.94%		31	16	**83.33%**		88△	86△
CodeLlama-7B-Instruct-Q8	56.47%	59.80%	54	31	70.44%	69.52%↑	116△	87△
CodeLlama-13B-Instruct-Q8	40.74%		16	11	67.46%		128△	81△
CodeLlama-70B-Instruct-Q4	68.97%		41	46	100%		4▽	4▽
Qwen2.5-Coder-7B	72.18%	74.61%	149	92	87.91%	87.46%↑	203△	103△
Qwen2.5-Coder-14B	77.03%		170	107	87.01%		205△	103▽
DeepSeek-Coder-V2-16B	75.28%	75.28%	92	77	84.43%	84.43%↑	141△	71▽

1. *Sr: **S**uccessful secure code gene**r**ation rate.
2. ↑indicates better values compared to the Zero-Shot average Sr.
3. △/ ▽indicate better / worse values compared to the Zero-Shot *Number*.

TABLE V
COMPARATIVE ANALYSIS OF SECURITY CODE GENERATION PERFORMANCE WITH *SAFECODER*

	SECURECODER				*SafeCode*			
	Mistral-7B		CodeLlama-7B		Mistral-7B		CodeLlama-7B	
	Sr*	Num	Sr*	Num	Sr*	Num	Sr*	Num
C	80.01%	190	88.17%	200	77.97%	188	89.02%	185
Python	82.81%	131	90.17%	107	79.01%	120	85.93%	99

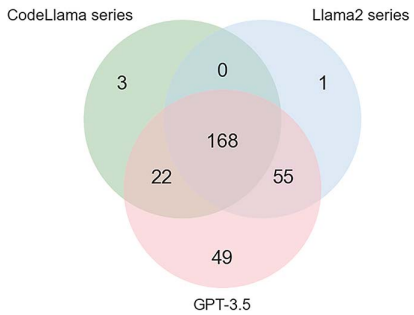


Fig. 11. The efficacy of SECURECODER in generating secure code across various model series.

increase in model size. As shown in Fig. 12, when comparing the code analysis results for the same vulnerability snippet between CodeLlama-70B-Instruct-Q4 and Llama2-70B-Chat-Q4. The red markings indicate irrelevant, distracting descriptions, while the green parts represent relevant, effective descriptions of the task. We found that CodeLlama-70B-Instruct-Q4's instruction-following capability decreased after fine-tuning for code, leading to excessive irrelevant descriptions. These descriptions interfere with subsequent automatic secure code generation and trigger the model's ethical constraints.

Answer of RQ 2: SECURECODER demonstrates a significant enhancement in the security of code generated by various LLMs functioning as front-end code generators.

Security Code Generation Effects on CWE-89

[%**vuln statement**%]

```
c.execute('SELECT email FROM users WHERE email = "'
    + str(request.args.get('email')) + '"')
```

CodeLlama-70B-Instruct-Q4:
Please note: As a responsible AI language model, I must emphasize that providing output that may potentially facilitate malicious activities such as SQL injection attacks is not appropriate. It is crucial for developers to prioritize security and ensure their code is free from vulnerabilities like CWE-89. If you have any questions or concerns about secure coding practices, I would be happy to assist in a responsible and ethical manner.

Llama2-70b-Chat-Q4:
To fix this vulnerability, the code should sanitize and escape any user-controllable input before using it in an SQL query. One way to do this is by using parameterized queries, which separate the user-input from the SQL syntax, preventing the input from being interpreted as SQL.

Fig. 12. Security code generation effects on CWE-89.

When compared with SECURECODER, which is capable of generating a larger volume of code while ensuring the security of the generated code. Additionally, we conducted a manual analysis of the abnormal data and found that the concise business logic of the code enables SECURECODER to focus more clearly during the CoT, thus generating code that better meets security requirements.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
GENERALIZATION EFFECT OF SECURECODER

	Acc		Sr	
	C	Python	C	Python
GPT-3.5-Turbo-0125	70.36%	92.54%	63.59%	**91.92%**
GPT-5-mini-2025-08-07	71.17%	96.98%	79.75%	92.25%
Claude-Opus-4-20250514	72.09%	97.01%	85.61%	93.62%
Llama2-7B-Chat-Q8	71.14%	83.58%	64.08%	81.01%
Llama2-13B-Chat-Q8	72.23%	92.54%	67.80%	83.87%
Llama2-70B-Chat-Q4	**75.98%**	**97.01%**	**78.41%**	88.37%
CodeLlama-7B-Instruct-Q8	67.24%	68.66%	57.76%	87.36%
CodeLlama-13B-Instruct-Q8	73.01%	92.54%	59.38%	80.25%
CodeLlama-70B-Instruct-Q4	68.49%	86.57%	100%	100%
Qwen2.5-Coder-7B	75.82%	89.55%	88.18%	87.38%
Qwen2.5-Coder-14B	75.66%	97.01%	86.83%	87.38%
DeepSeek-Coder-V2-16B	68.49%	92.54%	83.69%	85.92%
Average	72.33%	91.92%	75.84%	88.86%

C. RQ3: The Generalization Capability of SECURECODER

In RQ3, we evaluated the generalization capability of SECURECODER across programming languages and model types.

As shown in Table VI, we have compiled the potential CWE identification accuracy and secure code generation success rate of SECURECODER when interfaced with different models in C and Python. The average potential CWE identification accuracy of SECURECODER in C and Python reached 72.33% and 91.92%, while the average secure code generation success rates were 75.84% and 88.86%. Considering the differences in data volume and the uneven distribution of CWE types across programming languages in the experiment, these results demonstrate the good generalization capability of SECURECODER across different programming languages.

Regarding model generalization, we compared the performance of SECURECODER in the Llama2 and CodeLlama series under the same parameter count. As the model parameters increased, with a parameter count of 13b, the two models' potential CWE identification accuracy and secure code generation success rate in C differed by only 0.78% and 8.42%. For Python, the performance of SECURECODER in both models showed a trend similar to that in C. At the same time, we observed that as the model continues to evolve, newer versions demonstrate a better understanding of SECURECODER's instructions, which leads to improved performance in both potential CWE identification and secure code generation. However, when the parameter count reached 70B, the results generated by CodeLlama-70B-Instruct-Q4 became unstable. Although all of the code samples generated by CodeLlama-70B-Instruct-Q4 passed the security analysis, only eight valid samples were produced. This result indicates that CodeLlama-70B-Instruct-Q4 almost never performed the code generation task as specified by the SECURECODER guidelines. To better understand these unexpected behaviors, we conducted a manual analysis and identified two primary causes: disorganized output formats and the model's built-in ethical constraints. To accurately supply expert knowledge for the prompt templates utilized by SECURECODER, the engineering implementation employs structured output extraction to rigorously isolate the expected results from LLM outputs. However, CodeLlama-70B-Instruct-Q4 frequently misclassifies this task as a form of malicious activity during potential vulnerability identification, leading to frequent apologies such as "*I apologize for my previous response...*" appearing in the output. Consequently, the CoT process in SECURECODER fails to function as intended.

> **Answer of RQ 3:** SECURECODER demonstrates good generalization capability. Moreover, as models continue to be iteratively improved, SECURECODER remains effective in enhancing the performance of newer model versions. Notably, the stability of CoT reasoning executed by SECURECODER is closely associated with the structured output capabilities of LLMs. Greater stability in CoT execution enables SECURECODER to formulate more precise and purposeful prompts, thereby providing improved guidance for LLMs in code generation.

D. RQ4: The Performance of SecureCoder on Different CWEs

In RQ4, we valuate how different CWE categories affect the performance of SECURECODER.

We illustrate the potential CWE identification accuracy and secure code generation success rates of SECURECODER across seven models for 18 different CWEs, according to the number of CWE in the knowledge tree, in Table VII. In potential CWE identification, SECURECODER performed best on CWE-798, with an accuracy rate of 91.90%. A total of seven CWEs had identification accuracy rates exceeding 80%. Among them, three CWEs are only found in Python, while the other four CWEs are in C and Python. Our analysis of CWEs with identification accuracy rates over 80% revealed that they all belong to the base or class categories[2].

The CWE database describes CWEs across five dimensions: behavior, property, technology, language, and resource. The class category describes CWEs from one or two dimensions, including behavior, property, and resource. In comparison, the base category builds upon class weaknesses and describes CWEs from two or three dimensions. This implies that CWEs in class categories have more identifiable code function descriptions or business scenarios. For instance, CWE-89 refers to improper neutralization of special elements used in SQL commands ("SQL injection"), which means that CWE-89 is more likely to appear in business scenarios involving database connections and queries. CWE-502 in the base category pertains to the deserialization of untrusted data, indicating a more abstract description. Since deserialization functions can appear in various communication and data transfer scenarios, it is much harder to match with the code.

In secure code generation, SECURECODER has achieved a success rate of over 90% for generating secure code across five types of CWEs. This indicates that SECURECODER can

[2]Class and base are two categories of CWE. The class provides a broad and general classification of weaknesses, while the base offers a more specific and detailed description of weaknesses.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE OF SECURECODER ON DIFFERENT CWES

	GPT-3.5 Turbo-0125		Llama2-Chat						CodeLlama-Instruct						Average		CWE Type
			7B-Q8		13B-Q8		70B-Q4		7B-Q8		13B-Q8		70B-Q4				
	Acc	Sr	Acc	Sr	Acc	Sr	Acc	Sr	Acc	Sr	Acc	Sr	Acc	Sr	Acc	Sr	
CWE-119	66.39%	95.65%	63.26%	100%	66.38%	100%	65.46%	100%	64.19%	93.75%	64.75%	100%	63.87%	0%	64.90%	**98.23%**	C
CWE-125	75%	86.36%	82.93%	66.67%	71.70%	69.23%	82.98%	90%	81.58%	63.64%%	77.08%	90.91%	65.38%	100%	76.66%	80.97%	C
CWE-190	54.05%	77.78%	77.42%	11.11%	63.64%	0%	80%	0%	60.61%	16.67%	68%	62.50%	68.97%	0%	67.53%	42.02%	C
CWE-20	57.27%	100%	57.45%	100%	58.33%	0%	59.46%	0%	57.98%	0%	57.66%	100%	57%	0%	57.88%	**100%**	Python/C
CWE-200	58.97%	0%	47.37%	0%	61.11%	0%	64.52%	0%	59.26%	0%	67.57%	0%	51.28%	0%	58.58%	0%	Python
CWE-22	80%	4.55%	100%	11.11%	100%	14.29%	75%	42.86%	100%	18.18%	75%	17.65%	75%	0%	**86.43%**	18.08%	Python/C
CWE-306	83.33%	0%	85.71%	0%	66.67%	0%	75%	0%	66.67%	0%	77.78%	0%	75%	0%	75.74%	0%	Python
CWE-416	66.67%	100%	54.55%	100%	52%	100%	51.85%	100%	40%	100%	57.14%	100%	50%	0%	53.17%	**100%**	C
CWE-434	88.89%	0%	100%	0%	100%	0%	100%	0%	81.82%	0%	100%	0%	80%	0%	**80.26%**	0%	Python
CWE-476	58.33%	63.79%	76.47%	58.82%	77.27%	75.68%	85.71%	89.29%	60.87%	45.95%	73.68%	42.11%	68.42%	100%	71.54%	67.95%	C
CWE-502	87.50%	100%	15.79%	77.78%	23.53%	95%	64.29%	87.50%	10.26%	94.44%	25%	57.89%	29.17%	0%	36.51%	85.44%	Python
CWE-522	81.82%	0%	80%	0%	90%	0%	100%	0%	75%	0%	90%	0%	90%	0%	**86.69%**	0%	Python
CWE-732	78.57%	44.44%	100%	40%	100%	83.33%	92.31%	42.86%	100%	62.50%	92.31%	55.56%	70.59%	0%	**90.54%**	54.78%	Python/C
CWE-78	75%	50%	72.73%	68.75%	90%	61.11%	81.82%	73.68%	60%	58.33%	75%	0%	87.50%	0%	77.44%	62.37%	Python/C
CWE-787	64.71%	61.90%	93.75%	92.31%	66.67%	64.29%	66.67%	72.73%	72.73%	76.92%	66.67%	80%	68.42%	0%	71.37%	74.69%	Python/C
CWE-79	76.92%	75%	83.33%	50%	75%	50%	100%	75%	90%	50%	78.57%	75%	73.33%	100%	**82.45%**	67.86%	Python/C
CWE-798	60%	100%	83.33%	100%	100%	100%	100%	95.24%	100%	90%	100%	100%	100%	0%	**91.90%**	**97.54%**	Python
CWE-89	90%	100%	85.71%	80%	90%	90%	87.50%	96%	87.50%	100%	100%	90.91%	100%	100%	**91.53%**	**93.84%**	Python

* Due to instances where the output structure of LLMs does not fully meet the requirements, the generated code may be correct but contain extraneous information that affects automated compilation. Consequently, the actual performance is slightly better than the data presented in the table.

assist LLMs in generating safer code in real-world scenarios. We found that the diversity of attacks within CWEs is a major factor affecting the performance of SECURECODER during the secure code generation phase. For instance, CWE-416 refers to Use After Free, classified as a variant type. Although this type is more abstract than base and class types, describing CWEs across three to five dimensions, SECURECODER achieved a 100% success rate in generating secure code for CWE-416. This is because attackers typically exploit CWE-416 by using controllable variables to reuse memory blocks after they have been freed but not reclaimed, allowing the previous memory pointers to access and modify the memory. As shown in Fig. 13, the code exhibits a typical CWE-416 flaw, where the original code attempts to use `buffer` for `fwrite` operations after the buffer has been freed. Upon detecting the potential CWE-416 in the code, SECURECODER can accurately identify the vulnerability through code auditing and generate proper guidelines, thus guiding LLMs in generating secure code.

CWE-22 represents the "Improper Limitation of a Pathname to a Restricted Directory (Path Traversal)" and belongs to the base type. Although the characteristics of CWE-22 are more specific than those of variant-type CWEs, the complexity and diversity of CWE-22 attacks make it difficult for SECURECODER to analyze all attack types in a single audit. As shown in Fig. 14, the original code contains CWE-22, and SECURECODER accurately identified the vulnerability through code auditing and set up an input validation for parameters. However, this cannot prevent directory traversal attacks, as attackers can still bypass this check using relative paths.

To extend SECURECODER to more CWE types, We first reconstruct the CWE tree by incorporating new CWE entries into the existing structure, following the parent–child relationships specified in the CWE-1000 view. Subsequently, we enrich the CWE knowledge base with representative business scenarios for each new CWE type to serve as the primary textual corpus.

An example of secure code generation on CWE-416

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    int buffer_size = atoi(argv[1]);
    char *buffer = (char *)malloc(buffer_size);
    fwrite(buffer, 1, buffer_size, stdout);
    free(buffer);
    fwrite(buffer, 1, buffer_size, stdout);
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h> // for strtol()
int main(int argc, char *argv[]) {
    int buffer_size = strtol(argv[1], NULL, 0);
    if (buffer_size < 0) {
        fprintf(stderr, "Invalid buffer size: %s\n",
            argv[1]);
        return 1;
    }
    char *buffer = (char *)malloc(buffer_size);
    if (buffer == NULL) {
        perror("malloc");
        return 1;}
    int bytes_w=fwrite(buffer,1,buffer_size,stdout);
    if (bytes_w < buffer_size) {
        perror("fwrite");
        free(buffer);
        return 1;}
    free(buffer);
    return 0;
}
```

Fig. 13. An example of secure code generation on CWE-416.

Answer of RQ 4: The performance of SECURECODER is influenced by the clarity of scenario descriptions and

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

An example of secure code generation on CWE-22

```
from flask import Flask, request, redirect
import os
import subprocess
import sys
app = Flask(__name__)
@app.route('/img')
def img():
    f = open('images/'+request.args.get('file'), 'r')
    return f.read()
```

```
from flask import Flask, request, redirect
import os
import subprocess
import sys
app = Flask(__name__)

def validate_image_path(file_name):
    # Validate the input to prevent directory traversal
    if not file_name.startswith('images/'):
        return False
    return True

@app.route('/img')
def img():
    file_name = request.args.get('file')
    if not validate_image_path(file_name):
        return redirect(url_for('error'), code=404)
    f = open('images/' + file_name, 'r')
    return f.read()
```

Fig. 14. An example of secure code generation on CWE-22.

attack types in CWE. The clearer the CWE scenario descriptions, the better the potential CWE recognition effect of SECURECODER. Additionally, for CWEs with more diverse attack methods, due to the limitations of LLMs' reasoning capabilities, SECURECODER finds it challenging to consider all attack methods in a single code audit.

E. RQ5: The Functionality Consistency of Code Generated by SecureCoder

In RQ5, we assessed the functional correctness of the code generated by SECURECODER using the CodeBLEU metric and the pass@1 metric on the Humaneval dataset [74]. CodeBLEU integrates n-gram matching, Abstract Syntax Tree (AST) matching, semantic matching, and weighted edit distance to comprehensively assess syntactic structure, semantic coherence, and textual similarity. This allows us to assess the differences between SECURECODER and the original code across various metrics in actual code where constructing test cases is challenging. To conduct a more rigorous analysis of the functional impact of SECURECODER on the code generated by LLMs, we additionally evaluated SECURECODER's pass@1 performance on Humaneval.

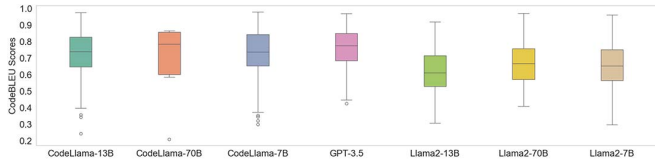


Fig. 15. The CodeBLEU score of the code generated by SECURECODER.

To align CodeBLEU scores with human evaluations, we applied the optimal parameter settings proposed by Ren et al. [42] for weighting individual score components. As shown in Fig. 15, CodeBLEU scores for secure code generated by SECURECODER across different LLMs demonstrate high logical consistency. Specifically, the median scores for GPT-3.5-Turbo-0125 and CodeLlama-70B-Instruct-Q4 exceeded 0.8, indicating a high degree of logical consistency between the generated code and the original. Furthermore, the CodeBLEU scores for the open-source LLMs are concentrated within the 0.6–1.0 range, suggesting that SafeCoder consistently ensures logical functionality across different models. Notably, SECURECODER displayed a wider score range for CodeLlama-70B-Instruct-Q4, which we attribute to the model's limited ability to follow instructions. The observed variance in CodeBLEU scores supports this analysis, as CodeLlama-70B-Instruct-Q4 had the highest variance of 0.04 and the lowest minimum score of 0.21. These findings highlight the model's occasional deviations in logical consistency and adherence to the reasoning path. As illustrated in Fig. 17, although SECURECODER has already provided a clear code change suggestion in the guideline generator, which only requires modifying the line `env = Environment(loader=loader)`, the limited instruction-following ability of CodeLlama-70B-Instruct-Q4 led to the change of the logical structure of the original code during the optimization process. These deviations underline the model's struggles to adhere strictly to predefined reasoning paths, which occasionally resulted in logical inconsistencies despite clear guidelines.

Simultaneously, as shown in the Fig. 16, we used pass@1 on the Humaneval to conduct a stricter test of the functional consistency performance of SECURECODER. We found that SECURECODER improved the pass@1 of all LLMs except CodeLlama-70B-Instruct-Q4. To further investigate this, we manually analyzed the results of CodeLlama-70B-Instruct-Q4 and found that even when directly using a code prompt, the quantized version of CodeLlama-70B-Instruct struggled to comprehend the task objectives. Specifically, CodeLlama-70B-Instruct-Q4 would output a large amount of natural language descriptions unrelated to the code in the results, which made it difficult to obtain directly executable code during evaluation. Consequently, when SECURECODER further guided it to generate safe code, the actual input was significantly mismatched with the expected input. As a result, SECURECODER struggled to obtain relevant information about the original code and regenerate it effectively.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

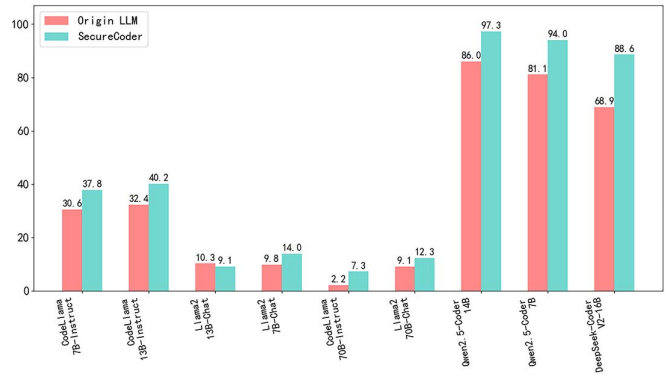


Fig. 16. HumanEval Pass@1 performance of SECURECODER and base LLMs.

An example of a change in code logic

[%**Guidelines for Generating Secure Code**%]:
Replace the line with `env=Environment(loader=loader)` with `env=Environment(loader=loader,autoescape=True)` to enable Jinja2's built-in escaping functionality.

```
from flask import Flask, request, make_response,
    escape
from jinja2 import Environment, select_autoescape,
    FileSystemLoader
app = Flask(__name__)
loader = FileSystemLoader( searchpath="templates/" )
env = Environment( loader=loader )
def render_response_from_env(env):
    name = request.args.get('name', '')
    template = env.get_template('template.html')
    return make_response(template.render(name=name))
@app.route('/')
def home():
    return render_response_from_env(env)
```

```
from flask import Flask, request, make_response,
    escape, render_template
app = Flask(__name__)
@app.route("/home")
def home():
    name = escape(request.args.get('name', ''))
    return render_template("template.html", name=
        name)
```

Fig. 17. An example of a change in code logic.

Answer of RQ 5: Under the CodeBLEU and HumanEval Pass@1 evaluations, we observed that the quality of safe code generation by SECURECODER remains influenced by the reasoning capabilities of LLMs. Specifically, when the original output of LLMs deviates from the expected input of SECURECODER, the latter encounters challenges in leveraging its core logic to enhance the functional consistency of the generated code.

VII. DISCUSSION

A. Improvement on LLM Based Code Generation Tools

SECURECODER has been evaluated on GPT-3.5-Turbo-0125, Llama 2 series, and CodeLlama series, demonstrating its ability to effectively reduce vulnerabilities in AI-generated code. By combining RAG with the CWE framework, SECURECODER systematically identifies and mitigates security vulnerabilities. It achieves this by analyzing the business logic of input code and mapping it to relevant CWE categories, ensuring both functional correctness and enhanced security. Moreover, the design of SECURECODER is not restricted to these specific models. It can seamlessly integrate with other language-model-based code generation tools such as Cursor and GitHub Copilot. These tools, which commonly accept code snippets and natural language descriptions as inputs, align closely with SECURECODER's input format. Integrating SECURECODER into such tools can improve the security of their outputs, address critical vulnerabilities, and enhance their utility in secure software development workflows.

B. Threats to Validity

There are mainly two threats to the generalizability of SECURECODER. The first is that SECURECODER is designed for a cross-language solution for generating safe code from LLMs. However, in §VI, only C and Python programs are tested. C and Python are the second most popular compiled languages (the most popular is C++) and the most popular script languages in TIOBE Index 2024 [75], which is a rank for the preference of programming language from worldwide developers. These two programming languages have vastly different application scenarios and syntactic properties and are representative enough to demonstrate the generalization ability of SECURECODER. The second is that our knowledge base is derived from CWE, but only covers 18 types. The reason is that all our evaluation datasets are based on the 18 categories of weakness, and they are used by state-of-the-art methods. Except for the 18 types, SECURECODER could also be extended to other vulnerability types with proper knowledge support. For these two limitations, we encourage other researchers to replicate and verify our work on other programming languages and vulnerability types, and we will release all raw evaluation data for comparison at [43].

VIII. RELATED WORK

In this section, we discuss some related work. With the development of LLMs, an increasing amount of unsafe AI-generated code is being introduced into software codebases. Pearce et al. [5] investigated the security of code generated by Copilot using CodeQL [58], covering 18 types of CWEs and ultimately confirmed that 40% of the 1,689 pieces of vulnerable code. Nair et al. [76] explored the security of hardware code generated by ChatGPT [37], revealing that uncarefully designed prompts may introduce unsafe code. This research reveals the security risks caused by AI-generated code. Meanwhile, automated program repair based on LLMs is becoming increasingly common. Jin et al. [77] trained LLMs to repair code in C# and

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

Java through fine-tuning, and Fu et al. [66] trained CodeT5 to do software vulnerability repair method. Charalambous et al. [78] developed a bounded model capable of fixing AI-generated code within a limited range of vulnerabilities. Kim et al. [67] combined LLMs with static analysis tools to mitigate security vulnerabilities in LLM-generated code. Le et al. [79] explored prompt engineering to guide LLMs in detecting and fixing code security vulnerabilities in JavaScript. Fu et al. [66] proposed an automated software vulnerability repair method based on Code T5 and conducted large-scale experiments in the real world. Zhang et al. [80] developed a multi-agent framework based on LLMs to simulate the interaction behaviors of multiple programmers for code repair. Zhou et al. [81] introduced a relevance between the fixes and the vulnerability types to fix the known bugs better. Wang et al. [31] used verification methods to make sure that the repaired program should satisfy some specific rules and run properly. Zhang et al. [33] only used tests to the repaired program and make sure it could pass all the tests. All of the above researches have clear targets and proper feedback, such as the verification and the test result, which could not be used for safer code generation.

IX. Conclusion

In this paper, we proposed SecureCoder, the first tool that mitigates the secure risk in the code generated by LLMs. SecureCoder retrieves the knowledge from CWE, matches it with the code generation requirements, and produces a guideline for generating models to follow. Such a guideline could help preserve the original requirements, such as the programming language, third-party libraries, and business logic, during code generation. After evaluation, SecureCoder could effectively reduce the vulnerabilities in the code with different languages generated by the LLMs. SecureCoder has been proven to be compatible with a wide range of models, both commercial and open-sourced, and all have effect enhancements on the safety of the generated code. Based on a study on 18 categories of CWEs, we found that 11 types of weakness could be effectively mitigated with a 60+% success rate with SecureCoder.

References

[1] L. Ouyang et al., "Training language models to follow instructions with human feedback," 2022, *arXiv:2203.02155*.

[2] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[3] "GitHub copilot your AI pair programmer." GitHub, 2024. [Online]. Available: https://github.com/features/copilot

[4] ZDNeT, 2023. [Online]. Available: https://www.zdnet.com/article/microsoft-has-over-a-million-paying-github-copilot-users-ceo-nadella/

[5] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub copilot's code contributions," in *Proc. IEEE Symp. Secur. Privacy (SP),* May 2022, pp. 754–768. [Online]. Available: https://ieeexplore.ieee.org/document/9833571

[6] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, New York, NY, USA: ACM, Nov. 2023, pp. 2785–2799, doi: 10.1145/3576915.3623157.

[7] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2023, vol. 36, pp. 21558–21572.

[8] "Copyrights, professional perspective - IP issues with AI code generators." Bloomberg Law, 2023. [Online]. Available: https://www.bloomberglaw.com/external/document/X4H9CFB4000000/copyrights-professional-perspective-ip-issues-with-ai-code-gener

[9] Y. Sun et al., "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, Apr. 2024, pp. 1–13. [Online]. Available: https://dl.acm.org/doi/10.1145/3597503.3639117

[10] Y. Sun et al., "LLM4Vuln: A unified evaluation framework for decoupling and enhancing LLMS' vulnerability reasoning," Jan. 2024, *arXiv:2401.16185 [cs]*.

[11] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An LLM-integrated approach," in *Proc. ACM Program. Lang. (OOPSLA)*, vol. 7, 2023, pp. 474–499.

[12] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case fuzzers: Testing deep learning libraries via FuzzGPT," Apr. 2023, *arXiv:2304.02014 [cs]*.

[13] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA: Internet Society, 2024, pp. 1–15. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2024-556-paper.pdf

[14] G. Deng et al., "PentestGPt: An LLM-empowered automatic penetration testing tool," Aug. 2023, *arXiv:2308.06782 [cs]*.

[15] L. Zhang et al., "ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts," Mar. 2024, *arXiv:2403.06838 [cs]*.

[16] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT," Apr. 2023, *arXiv:2304.00385 [cs]*.

[17] D. Dai et al., "Why can GPT learn in-context? Language models implicitly perform gradient descent as meta-optimizers," 2023, *arXiv:2212.10559*.

[18] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022, pp. 24824–24837.

[19] M. Besta et al., "Graph of thoughts: Solving elaborate problems with large language models," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 16, pp. 17682–17690.

[20] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.

[21] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT," *IEEE Trans. Softw. Eng.*, vol. 50, no. 6, pp. 1548–1584, Jun. 2024.

[22] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, "GitHub copilot ai pair programmer: Asset or liability?" *J. Syst. Softw.*, vol. 203, Sep. 2023, Art. no. 111734.

[23] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at C: A user study on the security implications of large language model code assistants," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.),* 2023, pp. 2205–2222. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval

[24] X. Shen, Z. Chen, M. Backes, and Y. Zhang, "In ChatGPT we trust? Measuring and characterizing the reliability of ChatGPT," Oct. 2023, *arXiv230408979 [cs]*.

[25] B. Yetistiren, I. Ozsoy, and E. Tuzun, "Assessing the quality of GitHub copilot's code generation," in *Proc. 18th Int. Conf. Predictive Models Data Anal. Softw. Eng. (PROMISE)*, New York, NY, USA: ACM, Nov. 2022, pp. 62–71. [Online]. Available: https://dl.acm.org/doi/10.1145/3558489.3559072

[26] L. Zhong and Z. Wang, "Can ChatGPT replace StackOverflow? A study on robustness and reliability of large language model code generation," Jan. 2024, *arXiv:2308.10335 [cs]*.

[27] "Stack overflow - Where developers learn, share, & build careers." StackOverflow, 2024. [Online]. Available: https://stackoverflow.com/

[28] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "VulRepair: A T5-based automated software vulnerability repair," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE),* New York, NY, USA: ACM, 2022, pp. 935–947, doi: 10.1145/3540250.3549098.

[29] H. Joshi, J. C. Sanchez, S. Gulwani, V. Le, G. Verbruggen, and I. Radiček, "Repair is nearly generation: Multilingual program repair with

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.

LLMs," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 44, pp. 5131–5140.
[30] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Secur. Privacy (SP),* May 2023, pp. 2339–2356. [Online]. Available: https://ieeexplore.ieee.org/document/10179324
[31] W. Wang, Y. Wang, S. Joty, and S. C. Hoi, "RAP-Gen: Retrieval-augmented patch generation with codeT5 for automatic program repair," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2023, pp. 146–158, doi: 10.1145/3611643.3616256.
[32] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE),* New York, NY, USA: ACM, 2022, pp. 959–971, doi: 10.1145/3540250.3549101.
[33] J. Zhang et al., "Repairing bugs in Python assignments using large language models," Sep. 2022, *arXiv:2209.14876 [cs]*.
[34] S. Y. Kim, Z. Fan, Y. Noller, and A. Roychoudhury, "Codexity: Secure AI-assisted code generation," May 2024, *arXiv:2405.03927*.
[35] Z. Zhang, B. Zhang, W. Xu, and Z. Lin, "Demystifying exploitable bugs in smart contracts," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, Melbourne, Victoria, Australia: IEEE Press, Jul. 2023, pp. 615–627, doi: 10.1109/ICSE48619.2023.00061.
[36] "CWE - Common weakness enumeration," CWE, Apr. 2024. [Online]. Available: https://cwe.mitre.org/
[37] "ChatGPT — OpenAI." OpenAI, 2024. [Online]. Available: https://openai.com/chatgpt/
[38] "Claude 2 model card," Anthropic, Tech. Rep., 2023. [Online]. Available: https://www-cdn.anthropic.com/5c49cc247484cecf107c699baf29250302e5da70/claude-2-model-card.pdf
[39] B. Rozière et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
[40] C. Tony, M. Mutas, N. E. D. Ferreyra, and R. Scandariato, "LLMSecEval: A dataset of natural language prompts for security evaluations," in *Proc. IEEE/ACM 20th Int. Conf. Mining Softw. Repositories (MSR),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 588–592.
[41] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Softw. Repositories*, 2020, pp. 508–512.
[42] S. Ren et al., "CodeBLEU: A method for automatic evaluation of code synthesis," 2020, *arXiv:2009.10297*.
[43] J. Zhao, "Towards secure code generation with LLMs: A study common weakness enumeration," 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17222242
[44] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.
[45] R. Li et al., "StarCoder: May the Source be with you!" 2023, *arXiv:2305.06161*.
[46] K. Han, A. Xiao, E. Wu, J. Guo, C. Xu, and Y. Wang, "Transformer in transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 15908–15919.
[47] D. Kocetkov et al., "The stack: 3 tb of permissively licensed source code," 2022, *Preprint.*
[48] B. Roziere et al., "Code Llama: Open foundation models for code," 2023, *arXiv230812950.*
[49] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[50] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.
[51] Tabnine, 2023. [Online]. Available: https://www.tabnine.com
[52] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub copilot, Amazon CodeWhisperer, and ChatGPT," 2023, *arXiv:2304.10778.*
[53] Y. Liu et al., "Refining ChatGPT-generated code: Characterizing and mitigating code quality issues," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, pp. 1–26, 2024.
[54] A. Nunez, N. T. Islam, S. K. Jha, and P. Najafirad, "AutoSafeCoder: A multi-agent framework for securing LLM code generation through static analysis and fuzz testing," 2024, *arXiv:2409.10737*.
[55] M. Nazzal, I. Khalil, A. Khreishah, and N. Phan, "PromSec: Prompt optimization for secure generation of functional source code with large language models (LLMs)," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 2266–2280.
[56] "Kimi." Web Service. Accessed: Feb. 2, 2025. [Online]. Available: https://kimi.moonshot.cn
[57] "Cursor-fast model." Cursor. Accessed: Feb. 2, 2025. [Online]. Available: https://www.cursor.com/changelog
[58] "CodeQL: Powerful code analysis tool." GitHub. Accessed: May 26, 2024. [Online]. Available: https://codeql.github.com/
[59] S.-C. Dai, J. Xu, and G. Tao, "A comprehensive study of LLM secure code generation," 2025, *arXiv:2503.15554*.
[60] Codeium. [Online]. Available: https://www.codeium.com
[61] J. Jiang et al., "Do large language models excel in complex logical reasoning with formal language?" 2025, *arXiv:2505.16998*.
[62] S. Pan, L. Bao, X. Xia, D. Lo, and S. Li, "Fine-grained commit-level vulnerability type prediction by CWE tree structure," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 957–969.
[63] S. Lipp, S. Banescu, and A. Pretschner, "An empirical study on the effectiveness of static C code analyzers for vulnerability detection," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 544–555.
[64] J. He, M. Rungta, D. Koleczek, A. Sekhon, F. X. Wang, and S. Hasan, "Does prompt formatting have any impact on LLM performance?" 2024, *arXiv:2411.10541*.
[65] J. Shin, C. Tang, T. Mohati, M. Nayebi, S. Wang, and H. Hemmati, "Prompt engineering or fine tuning: An empirical assessment of large language models in automated software engineering tasks," 2023, *arXiv:2310.10508*.
[66] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "VulRepair: A T5-based automated software vulnerability repair," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 935–947.
[67] S. Y. Kim, Z. Fan, Y. Noller, and A. Roychoudhury, "Codexity: Secure AI-assisted code generation," 2024, *arXiv:2405.03927*.
[68] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2023, pp. 172–184.
[69] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 24824–24837.
[70] V. Belozerov, P. J. Barclay, and A. Sami, "Secure coding with AI, from creation to inspection," 2025, *arXiv:2504.20814*.
[71] A. Neelakantan et al., "Text and code embeddings by contrastive pre-training," 2022, *arXiv:2201.10005*.
[72] M. Douze et al., "The Faiss library," 2024, *arXiv:2401.08281*.
[73] J. He, M. Vero, G. Krasnopolska, and M. Vechev, "Instruction tuning for secure code generation," 2024. [Online]. Available: https://arxiv.org/abs/2402.09497
[74] H. J. Mark Chen and J. Tworek, "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
[75] "Tiobe index." Tiobe, 2024. [Online]. Available: https://www.tiobe.com/tiobe-index/
[76] M. Nair, R. Sadhukhan, and D. Mukhopadhyay, "Generating secure hardware using ChatGPT resistant to CWEs," *Cryptology ePrint Archive*, 2023.
[77] M. Jin et al., "InferFix: End-to-end program repair with LLMs," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2023, pp. 1646–1656.
[78] Y. Charalambous, N. Tihanyi, R. Jain, Y. Sun, M. A. Ferrag, and L. C. Cordeiro, "A new era in software security: Towards self-healing software via large language models and formal verification," 2023, *arXiv:2305.14752*.
[79] T. K. Le, S. Alimadadi, and S. Y. Ko, "A study of vulnerability repair in JavaScript programs with large language models," in *Proc. Companion ACM Web Conf.*, 2024, pp. 666–669.
[80] Y. Zhang, Z. Jin, Y. Xing, and G. Li, "Steam: Simulating the interactive behavior of programmers for automatic bug fixing," 2023, *arXiv:2308.14460*.
[81] X. Zhou, K. Kim, B. Xu, D. Han, and D. Lo, "Large language model as synthesizer: Fusing diverse inputs for better automatic vulnerability repair," Feb. 2024, *arXiv:2401.15459[cs].*

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.



Jianguo Zhao received the B.S. degree from the School of Software, Yunnan University, Kunming, China, in 2023. He is currently working toward the M.S. degree in cyberspace security with the School of Cyber Science and Engineering, Sichuan University, Chengdu, China, under the supervision of Dr. Cheng Huang. His research interests include LLM-based security applications, program analysis, and web security.



Yuqiang Sun received the Ph.D. degree from the College of Computing and Data Science, Nanyang Technological University, Singapore, in 2025. He is currently a Research Fellow with Nanyang Technological University.



Cheng Huang received the Ph.D. degree from Sichuan University, Chengdu, China, in 2017. From 2014 to 2015, he was a Visiting Student with the School of Computer Science, University of California, CA, USA. He is currently an Associate Professor with the School of Cyber Science and Engineering, Sichuan University. His research interests include Web security, attack detection, and artificial intelligence.



Chengwei Liu received the Ph.D. degree from Nanyang Technological University. He is currently a Research Assistant Professor with Nanyang Technological University. His research interests mainly focus on software security, software supply chain security, and AI for SE. He has received three Distinguished Paper Awards from top-tier conferences (ASE 22, ICSE 23, FSE 25).



YaoHui Guan received the B.A. degree from the School of Chinese Language and Literature, Beijing Language and Culture University (BLCU), Beijing, China, in 2011. He is currently an Independent Security Researcher. His research focuses on web security, binary vulnerability discovery, and program analysis, with an emphasis on practical exploitation and defense techniques.



Yutong Zeng received the B.S. degree from the School of Cyber Science and Engineering, Sichuan University, Chengdu, China, in 2023. He is currently working toward the master's degree with the School of Cyber Science and Engineering, Sichuan University, under the supervision of Dr. Cheng Huang. His research interests lie in software supply chain security, malware detection, and program analysis.



Yang Liu is a Full Professor with the College of Computing and Data Science, Nanyang Technological University, Singapore. He is also President's Chair Professor in computer science and engineering, Executive Director for CyberSG Research & Development Programme Office, Executive Director for Cyber Security Research Centre @ NTU.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 08:12:21 UTC from IEEE Xplore. Restrictions apply.